星河工坊 · 部署文档

安装、演示、备份、独立还原与故障处置

版本：1.0 / 2026-10-19 / 默认仅限本机访问；成员编号 DEMO01、DEMO02、DEMO03 为待替换的演示占位编号。

1. 前提与边界

Windows/macOS 安装并启动 Docker Desktop，选择 Linux containers；Linux 使用 Docker Engine 与 Compose。要求 Compose 2.24+、Python 3.12+，首次构建需要网络。PHP 8.4、Yii2 2.0.55 与 MySQL 8.4 的精确镜像及包版本由仓库锁文件确定。

所有命令从仓库根目录执行。无需全局 PHP 或 MySQL。镜像复制源码，修改 PHP、PDF 或 SQL 后要重新构建应用镜像；已有数据卷不因重建而自动重新导入种子。

2. 一键启动

运行：python scripts/dev.py up

该命令构建容器、等待数据库健康并执行检查。首次空数据库由 scripts/install.php 加载 data/schema.sql；已有数据库保留业务数据。data/install.sql 是实际导出，含 DROP TABLE，不在每次启动时导入。然后打开 http://127.0.0.1:8080/，进入 /admin 使用 admin / admin123456 登录。

等价容器命令：docker compose up --build --wait --wait-timeout 240

端口冲突时复制 .env.example 为 .env，设置 APP_PORT 为未占用端口后重启。默认只绑定 127.0.0.1，数据库不开放宿主端口。不要把演示密码或默认配置用于公网服务。

3. 可选原生 PHP 开发

运行 python setup.py 初始化项目内 PHP 与 Composer；Windows 使用 . .\activate.ps1，Bash/Zsh 使用 source ./activate.sh。也可使用 python scripts/php.py 直接运行 PHP。

原生 PHP 不会自动穿透 Compose 私有网络；必须先提供可达 MySQL，并设置 DB_HOST、DB_PORT、DB_DATABASE、DB_USERNAME、DB_PASSWORD。然后运行 python scripts/serve.py --port 8080，脚本安装后启动同一网站。课程演示优先采用完整 Compose，不把“PHP 可运行”误作“数据库可访问”。

4. 演示验收流程

顺序	操作	核对结果
1	启动服务并打开首页	页面可读、数据库健康，中文及统计正常。
2	下载六项课程 PDF	文件完整可读，文件名与页面标题对应。
3	访客提交独特留言	收到待审核提示；访客窗口不提前公开。
4	管理员登录并批准、回复	刷新访客页后留言及回复可见。
5	编辑团队或项目信息	刷新公开页面看到修改；越权操作被拒绝。
6	退出与再次访问后台	需重新登录；直接构造地址不能绕过授权。
7	执行重建与独立还原	内容保留，重新登录能继续管理。

5. 备份

运行：python scripts/dev.py export --path .cache/exports/team-backup.sql

备份通过 docker compose cp 保留原始 SQL 字节，避免 PowerShell 文本重定向编码破坏中文。输出在项目内 .cache，不应把含真实留言、密码哈希或个人资料的备份提交到 Git。

InnoDB 单事务（Single Transaction）导出适合无并发表结构变更的场景；不要在导出时执行迁移或 ALTER TABLE。SQL 逻辑备份不包含 Docker 配置、用户权限、会话和外部文件。数据卷本身不是备份，应把审核后的备份保存在独立可靠介质。

6. 独立还原演练

先导出备份并确认文件非空。以下操作使用不同 Compose 项目名与端口，隔离原数据库卷；建议在新的终端会话执行，结束后清除所设环境变量。

PowerShell：$env:COMPOSE_PROJECT_NAME='xinghe-restore'; $env:APP_PORT='8081'

Bash/Zsh：export COMPOSE_PROJECT_NAME=xinghe-restore APP_PORT=8081

运行：python scripts/dev.py up

在新环境运行：python scripts/dev.py restore --path .cache/exports/team-backup.sql --confirm-restore

警告：导入会覆盖目标数据；必须确认当前项目是 xinghe-restore，而不是原演示环境。该命令不会自动为目标再做备份。

运行：python scripts/dev.py check

打开 http://127.0.0.1:8081/，核对独特留言、回复、成员与项目内容，登录后台再完成一次编辑，确认恢复的是业务状态而非仅建表成功。

完成后运行 python scripts/dev.py down。若明确不再保留演练数据，可以只在确认项目名为 xinghe-restore 的终端执行 docker compose down --volumes；不要在原环境误执行。关闭演练终端即可避免环境变量污染正常演示。

7. 持久化、停机与破坏性操作

运行 python scripts/dev.py persistence 检查重建后的持久化；运行 python scripts/dev.py down 正常停止并保留卷。会话在 /tmp，容器重建后退出登录是预期行为，不代表业务数据丢失。

docker compose down --volumes 会永久删除当前项目数据库卷。修改 MYSQL 初始化变量不会更新既有数据库账户；不能通过反复清卷“修复”普通配置错误。恢复前先核对项目名、端口、备份时间和目标数据重要性。

8. 故障排查与交付

查看日志：python scripts/dev.py logs。重建后仍不可用时检查 Docker 引擎、镜像拉取、端口占用和数据库健康。不要禁用 TLS 或校验散列来绕过下载错误。

提交 SQL 只保留课程可公开的演示数据。六份 PDF 在 data/team，编辑源在 docs/course；运行 python scripts/build_submissions.py 可重建（需要 ReportLab 与中文字体，--font 可指定 TrueType 字体）。正式学号变更时需同步脚本、下载记录与文件名。

公网部署还需要 HTTPS、专用 Web 服务器、真实密钥管理、全面限速、备份策略与安全审查；本地演示不等于生产部署。登录失败已有数据库级限速，留言已有会话级防重复提交，仍不能代替公网反滥用策略。

参考：MySQL mysqldump，dev.mysql.com/doc/refman/8.4/en/mysqldump.html；Docker readiness，docs.docker.com/compose/how-tos/startup-order/。